MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)
Mini-Project Title: Mini-Project 3 — Receipt OCR & Expense Tracker
Student Name: Trần Kim Bá Triều
Student ID: 23IT287
Submission Date: [DD/MM/YYYY]

1. GENERAL INFORMATION & DELIVERABLE LINKS

Team Members:

* **Trần Kim Bá Triều** — Student ID: **23IT287** — Role: **Individual Project Developer** — Contribution: **100%**

🔗 **Live Demo URL (Web):** https://receipt-ocr-tracker.web.app *(Firebase Hosting; IndexedDB SQLite persistence)*
Mirror (Netlify): https://delightful-rabanadas-6fa037.netlify.app *(Password:* `My-Drop-Site`*)*
💻 **GitHub Repository:** https://github.com/Trieuoiu123/miniP3.git
📱 **Android APK Package:** `build/app/outputs/flutter-apk/app-release.apk` (82.7 MB, release build verified)
🎥 **Video Demo (Optional):** [Not available]

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details
1	**Receipt Image Capture & Selection**	Complete	Integrates `image_picker` for direct camera snapshot and photo gallery selection on Android; invokes native web file selector on Flutter Web.
2	**On-Device OCR Text Recognition**	Complete	Uses `google_mlkit_text_recognition` for 100% offline Latin script extraction on Android; provides an adaptive non-blocking fallback routing directly to manual entry on Web.
3	**Automatic Receipt Data Parsing**	Complete	Custom regex heuristics in `receipt_parser_service.dart` automatically detect store title, receipt date (`dd/MM/yyyy`), and total bill amount (`TỔNG CỘNG` , `GRAND TOTAL`).
4	**Receipt Review & Verification Screen**	Complete	`review_receipt_screen.dart` provides form validation, custom Gregorian date picker, collapsible raw OCR text viewer, and editable fields before saving.
5	**Local Offline Data Persistence**	Complete	Native Android stores relational records in SQLite via `sqflite` . Flutter Web utilizes `sqflite_common_ffi_web` with `sqlite3.wasm` backed by browser IndexedDB across reloads.
6	**Expense History & Full CRUD**	Complete	Descending chronological ledger in `receipt_history_screen.dart` with instant search filter, transaction detail inspection modal, and delete action with confirmation dialog.
7	**Dashboard & Data Visualization**	Complete	`dashboard_screen.dart` renders real-time monthly spend KPIs and a zero-dependency `CustomPainter` bar chart supporting dual analytical modes (7 days and 6 months).
8	**Cross-Platform Build & Deployment**	Complete	Android release APK built via Gradle (82.7 MB); Web bundle compiled in 28.7s and successfully deployed to Google Firebase Hosting with global CDN.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

3.1. System Overview

The **Receipt OCR & Expense Tracker** application automates expense recording by digitizing paper receipts into structured financial records. The operational workflow follows six sequential stages:

```
[1. Select Image] ──> [2. On-Device OCR] ──> [3. Regex Heuristics]
                                │                       │
[6. Dashboard Chart] <── [5. SQLite DB] <── [4. Review & Edit Form]
```

3.2. Technology Stack

Framework: Flutter `3.41.0` (Dart `3.11.0`), Material Design 3 (Emerald `#0F766E` / Amber `#D97706`).

Hardware & OCR: `image_picker: ^0.8.13+17`, `google_mlkit_text_recognition: ^0.16.0`.

Database Engine: `sqflite: ^2.4.2+1`, `path_provider: ^2.1.5` (Android); `sqflite_common_ffi_web: ^1.1.1` (Web).

Data Visualization: Pure Flutter `CustomPainter` API (zero external charting packages).

Deployment & Cloud: Google Firebase Hosting (Production CDN) and Netlify Drop.

3.3. Project Directory Structure

```
receipt_ocr_expense_tracker/
├── lib/
│   ├── main.dart                      # Entry point, WidgetsBinding, Web SQLite factory setup
│   ├── app.dart                       # MaterialApp theme and navigation configuration
│   ├── core/                          # Constants, M3 themes, date & currency formatters
│   │   └── widgets/app_image_view.dart# Conditional export: Image.file (IO) vs Image.network (Web)
│   ├── models/receipt.dart            # Immutable entity with copyWith, toMap, fromMap (int VND)
│   ├── data/                          # SQLite helper (AppDatabase) & ReceiptRepository
│   ├── services/
│   │   ├── receipt_image_service.dart # Conditional export: disk copy (IO) vs Base64 Data URL (Web)
│   │   ├── text_recognition_service.dart # Conditional export: ML Kit (IO) vs Web fallback
│   │   └── receipt_parser_service.dart# Heuristic regex engine (Merchant, Date, Grand Total)
│   └── features/
│       ├── dashboard/                 # KPI summary card & CustomPainter expense bar chart
│       ├── scan/                      # Image selector, progress indicator, OCR thumbnail preview
│       ├── review/                    # Form validation, DatePicker, raw OCR inspector
│       └── history/                   # Searchable history ledger & detail deletion modal
├── test/                              # 24 automated unit & widget tests (100% passing)
└── web/                               # Web worker (sqflite_sw.js), sqlite3.wasm, SPA redirects
```

3.4. Data Processing Flow

1. **Acquisition:** Receipt photo is selected via `ImagePicker`.
2. **Text Recognition:** Native Android invokes Google ML Kit offline text recognizer to extract raw text blocks.
3. **Regex Extraction:** `ReceiptParserService` searches for high-priority total keywords (`TỔNG CỘNG`, `GRAND TOTAL`, `THANH TOÁN`), formats dates into `dd/MM/yyyy`, and extracts store titles while filtering noise tokens.
4. **Human Verification:** Candidate fields populate `ReviewReceiptScreen` where users verify and modify data.
5. **Database Commit:** Validated records are inserted into SQLite via parameterized queries.

3.5. Storage, State Management & Platform Compatibility

State Management: Idiomatic Flutter `StatefulWidget` with structured `setState()`, encapsulated service facades, and typed route returns ensuring predictable UI synchronization without heavyweight bloat.

Relational Storage: Stores currency as integer values (`INTEGER` in SQLite) to avoid floating-point inaccuracies.

Platform Adaptation: Decoupled via **Dart Conditional Imports**. Native Android reads local files and executes ML Kit OCR; Flutter Web serializes images into persistent Base64 Data URLs and drives an in-browser SQLite database via WebAssembly (`sqlite3.wasm`) and `IndexedDB`.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

4.1. Visual Inspection Screenshots

📷 Screenshot 1: Dashboard & CustomPainter Expense Bar Chart

The central dashboard displaying current month financial KPIs (total spend in VND, receipt counter), quick-access navigation buttons, and the custom-painted 7-day/6-month interactive expense chart.

```
!Dashboard Screen
```

> **[Screenshot required: Dashboard screen with monthly summary card and CustomPainter bar chart]**

> *Caption:* Primary landing dashboard displaying aggregated monthly expense metrics and dual-mode CustomPainter bar chart. Verified live on Android release APK and Firebase Web deployment.

📷 Screenshot 2: Receipt Scanning & Review Form Verification

The receipt verification form displaying image preview, automatically parsed candidate fields (Merchant, Date, Total Amount), Gregorian calendar picker, and collapsible raw OCR comparison drawer.

```
!Receipt Review Screen
```

> **[Screenshot required: Receipt review form displaying extracted candidate fields and edit inputs]**

> *Caption:* Verification screen enforcing human-in-the-loop validation, date parsing sanity checks, and manual field editing prior to SQLite database persistence.

📷 Screenshot 3: Expense History Ledger & Detail Deletion Modal

The searchable historical ledger displaying chronologically sorted transaction items, real-time merchant title query filter, and transaction detail inspection modal with protected delete confirmation.

```
!Expense History Screen
```

> **[Screenshot required: Expense history transaction list with search filter and detail modal]**

> *Caption:* Transaction history screen showing real-time query filtering, descending date sorting, and detail inspection modal. Verified with full CRUD database operations.

4.2. Verification & Automated Test Summary

Static Code Analysis (`flutter analyze`): `No issues found!` (0 errors, 0 warnings across all Dart files).

Automated Test Suite (`flutter test`): 24/24 unit & widget tests passed 100% (testing currency formatters, Gregorian calendar boundaries, regex parser scenarios, data models, and widget navigation flows).

Native Android Release (`flutter build apk --release`): Built successfully in 89.2s; output artifact verified at `build/app/outputs/flutter-apk/app-release.apk` (**82.7 MB**).

Web Build & Cloud Deployment: Web bundle compiled in 28.7s; deployed to Firebase Hosting at **`https://receipt-ocr-tracker.web.app`** with verified HTTP 200 uptime and IndexedDB data persistence.

5. TECHNICAL CHALLENGES & RESOLUTIONS

5.1. Challenge 1: Cross-Platform `dart:io` & SQLite Incompatibility on Web

Challenge: Compiling standard Flutter code for Web caused fatal compilation errors due to `dart:io` references (`Platform`, `File`) in `Image.file` and `sqflite` native C bindings. Furthermore, browser `blob:` URLs expire upon page reload, causing receipt photos to disappear.

Cause: Web browsers operate in a sandboxed JavaScript/WebAssembly context lacking access to local filesystem paths and native desktop C shared libraries.

Resolution:

1. Implemented **Dart Conditional Imports** (`app_image_view.dart`, `receipt_image_service.dart`, `text_recognition_service.dart`) to conditionally bind platform-specific implementations at compile time.
2. Integrated `sqflite_common_ffi_web` coupled with `sqlite3.wasm` and a dedicated Web Worker (`sqflite_sw.js`), configuring `databaseFactory = databaseFactoryFfiWeb` in `main.dart` to persist data in browser **IndexedDB**.
3. Replaced temporary blob URLs on Web by encoding picked files into **Base64 Data URLs** (`...`) stored directly in SQLite, guaranteeing 100% visual persistence across browser refreshes (F5).

Verification: Successfully executed `flutter build web --release` without errors. Verified on live Chrome browser: receipts, dates, amounts, and images persist reliably across hard page refreshes.

5.2. Challenge 2: Cross-Drive Incremental Compilation Failure in Windows Gradle

Challenge: Building the Android APK failed during Gradle execution with `IllegalArgumentException: this and base files have different roots: E:\... and C:\...`.

Cause: The project directory is located on root drive `E:`, whereas Android Gradle caches, Kotlin daemon, and Flutter Pub Cache are stored on root drive `C:`. The Kotlin Gradle Plugin's incremental compilation mechanism cannot compute relative paths across different Windows drive roots.

Resolution:

1. Updated `android/gradle.properties` with non-incremental compilation flags:

```
properties
kotlin.incremental=false
kotlin.incremental.useClasspathSnapshot=false
```

2. Added ProGuard rules (`-dontwarn com.google.mlkit.vision.text.**`) to suppress harmless R8 shrinking warnings for unused Asian script classes bundled in ML Kit base libraries.
3. Configured Java runtime environment to Microsoft OpenJDK 17 (`flutter config --jdk-dir`).

Verification: `flutter build apk --release` compiled cleanly in 89.2s without exceptions, generating the production release APK package (`app-release.apk`, 82.7 MB).

5.3. Conclusion

The **Receipt OCR & Expense Tracker** project has been successfully completed, meeting all core functional requirements with verified dual-platform support. The mobile application operates 100% offline on Android devices with private on-device machine learning OCR, while the web edition provides full analytical and CRUD capabilities with in-browser WebAssembly SQLite persistence. Future enhancements may explore lightweight on-device multi-language OCR models and automated cloud backup synchronization.